FEAT-ZD-48600 · DE-06 Data Mapping · Updated data mappings and transformation steps

Data Engineering agent · item feat.mappings · source hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/feat.mappings.md · approved by Data engineer sim.data_engineer

feat.mappings: FEAT-ZD-48600

These are the updated data mappings and transformation steps for the Data Engineering lane of FEAT-ZD-48600 ("Show which overdue invoices belong to vendors that usually pay late") (record:brief; feat.data_model_change header). This is step DE-06, Data Mapping. The workbook action is "Update the data mapping based on the data model and fetch the latest data mappings based on the customer input". The step has no human gate: automated checks, the independent judge and sampled human evaluation review it (record:brief). The steps that read feat.mappings next are DE-07 (with task.components) and DE-08 Coding (file:hobasa-data-engineering-agent/.claude/skills/data-engineering-steps/SKILL.md).

Field	Value	Source
Trace ID	FEAT-ZD-48600	record:brief
Step	DE-06 Data Mapping	record:brief
Owner	Data engineer	record:brief
Contracted inputs	feat.data_model_change (commit 3943254); org.customer_mappings (commit e8afce8, feature org-override)	record:brief
Background read	feat.design (722a98f): B1.1, B1.2, B1.4, B1.5, HC-3, DC-1, DC-2, DD-6, DD-7, NF-10, SE-4	journey:feat.design
Code read	ap_pipeline `ingest.py`, `transform.py`, `dq.py`, `pipeline.py`, `schedule.json`, `tests/test_pipeline.py` (step tool view); `data/sample_feed.csv` (checkout)	tool:repo; file:ap-pipeline-sample/data/sample_feed.csv
Latest customer contract	Vendor feed v3 columns: `invoice_id, vendor, due_date, amount, paid, paid_date`. `paid_date` is optional, an ISO date, and empty when unpaid. Mapping: `paid_date -> Invoice.paid_date`. Schedule: daily 02:00 Asia/Kolkata (unchanged)	org.customer_mappings
Result	**One feed-to-Invoice column mapping is added:** `paid_date -> Invoice.paid_date`**. The existing column mappings (**`invoice_id`**,** `vendor`**,** `due_date`**,** `amount`**,** `paid`**) are unchanged. The data model change also brings one new reject reason,** `'paid_date'`**, and one derived, in-memory view, the Delay history, mapped in section 4. The customer contract agrees with the Data Model change on every field, with one wording difference raised as MQ-2. The schedule stays daily 02:00 Asia/Kolkata, the same as** `schedule.json`**.** There is no new source, no new stored entity, no storage change and no schedule change	org.customer_mappings; feat.data_model_change (sections 1, 2, 3.1, 3.4, 4, 6); tool:repo (`schedule.json`, `transform.py`)

1. Latest customer mapping contract compared with the Data Model change

Item	Customer contract (org.customer_mappings)	Data Model change (feat.data_model_change)	Agreement	Source
Feed version	v3	Feed v2 → v3 adds column `paid_date`; feed v2 must keep loading	Agrees	org.customer_mappings; feat.data_model_change (sections 1, 2)
Columns	`invoice_id, vendor, due_date, amount, paid, paid_date`	v2 columns `invoice_id,vendor,due_date, amount,paid`, plus `paid_date`	Agrees, same names and order	org.customer_mappings; feat.data_model_change (section 2)
`paid_date` column name	`paid_date`	`paid_date`; taken from the data model and design because no v3 specification was in the inputs (CQ-DE4-4)	Agrees. The customer contract now names the column, so CQ-DE4-4 can close (MQ-1)	org.customer_mappings; feat.data_model_change (sections 3.2, 7)
`paid_date` format	Optional, ISO date, empty when unpaid	ISO date, parsed with `date.fromisoformat` after stripping whitespace; missing, `None` or blank → `None`	Agrees	org.customer_mappings; feat.data_model_change (section 3.2)
Target	`paid_date -> Invoice.paid_date`	Invoice `paid_date (date or None)`, added in feed v3	Agrees	org.customer_mappings; feat.data_model_change (section 3.1)
Unpaid row that carries a `paid_date`	The column is "empty when unpaid"	Accepted and the date kept (interim, DD-7, pending OQ-B6)	Wording difference, raised as MQ-2. This mapping follows the Data Model change meanwhile	org.customer_mappings; feat.data_model_change (sections 3.2, 7)
Schedule	Daily 02:00 Asia/Kolkata (unchanged)	Unchanged: daily, 02:00, Asia/Kolkata	Agrees, and matches `schedule.json` (section 5)	org.customer_mappings; feat.data_model_change (section 4); tool:repo (`schedule.json`)

2. Source-to-target mapping: feed row → Invoice record

Source: one row of the vendor invoice feed (CSV text with a header row). `ingest.parse_feed` turns it into a `dict` of strings keyed by the header. Target: the Invoice record, a plain `dict`, that `transform.to_model` returns (tool:repo `ingest.py`, `transform.py`; journey:feat.design DC-1).

Source column	Target attribute	Target type	Nullable	Transformation rule	Change	Source
`invoice_id`	`id`	str	No	`strip()`; missing column → row rejected (`KeyError`)	None	tool:repo (`transform.py`); feat.data_model_change (sections 5, 6)
`vendor`	`vendor`	str	No (see DMQ-1, carried)	`strip()`; missing column → row rejected (`KeyError`)	None	tool:repo (`transform.py`); feat.data_model_change (sections 3.1, 6, 7)
`due_date`	`due_date`	date	No	`date.fromisoformat(...)` on the raw value (no strip today); invalid → row rejected (`ValueError`)	None	tool:repo (`transform.py`); feat.data_model_change (section 6)

Source column	Target attribute	Target type	Nullable	Transformation rule	Change	Source
`amount`	`amount`	Decimal, 2 dp	No	`Decimal(...)`, never float; 2 dp is not enforced today (DMQ-3, carried)	None	tool:repo (`transform.py`); feat.data_model_change (sections 3.1, 5, 7); record:memory (D-2026-044, L-011)
`paid`	`paid`	bool	No	`r.get("paid", "false").strip().lower() == "true"`: the string `"true"` in any case → true; any other string → false; missing column → false; a `None` cell (short row) raises `AttributeError` and rejects the row	None	tool:repo (`transform.py`); feat.data_model_change (section 6)
`paid_date` (v3, optional)	`paid_date`	**date**	**Yes (`None`)**	Missing column (feed v2), `None` (short row) or blank → `None`. Otherwise `date.fromisoformat(value.strip())`. A non-blank value that is not an ISO date rejects the row with reason `'paid_date'`	**New**	org.customer_mappings; feat.data_model_change (sections 3.2, 6); journey:feat.design (B1.2, B1.4)

3. Transformation steps (per feed row, in this order)

The order keeps today's behaviour for the existing fields and maps `paid_date` last, so an existing reject reason wins over `'paid_date'` (feat.data_model_change section 3.2 "Precedence"; journey:feat.design DD-6).

1. **Parse** the CSV text into rows keyed by the header (`ingest.parse_feed`, unchanged). A v3 header carries `paid_date` into each row; a v2 header has no `paid_date` key (tool:repo `ingest.py`; feat.data_model_change section 2).
2. **Map the existing fields** `id`, `vendor`, `due_date`, `amount` and `paid` exactly as today, inside the existing `try`. On `KeyError`, `ValueError`, `InvalidOperation` or `AttributeError`, append `{"row": row, "reason": <exception type name>}` to the rejected list and go to the next row (tool:repo `transform.py`; journey:feat.design B1.2 step 1).
3. **Map** `paid_date` only for a row that passed step 2: read `row.get("paid_date")`; missing, `None` or blank gives `None`; otherwise parse the stripped value with `date.fromisoformat` (private helper `_parse_paid_date`) (journey:feat.design B1.2 step 2; feat.data_model_change section 3.2).
4. **Reject an invalid** `paid_date`: if the parse in step 3 raises `ValueError`, append `{"row": row, "reason": "paid_date"}` (constant `PAID_DATE_REASON`) and add no record (journey:feat.design B1.2 step 2; feat.data_model_change section 3.3).
5. **Emit the record** with the keys `id`, `vendor`, `due_date`, `amount`, `paid` and `paid_date`. Every accepted record carries `paid_date` (a `date` or `None`), including feed v2 rows (feat.data_model_change sections 3.2, 4; journey:feat.design B1.2 step 3).

6 **Do not cross-validate** `paid` **and** `paid_date`: a paid record without `paid_date` is kept; an unpaid row with a valid `paid_date` is accepted and keeps the date (interim, DD-7, pending OQ-B6) (feat.data_model_change section 3.2; journey:feat.design DD-7).

7 **Data-quality report**, no code change: `quality_report` puts the sorted set of rejected `reason` values in `reject_reasons`, so `'paid_date'` appears whenever such a row is rejected (tool:repo `dq.py`; feat.data_model_change section 3.3).

8 **Derive the Delay history** with the new function `delays.delays_by_vendor(records)` in a new module (TSK-02), which is pure and in-memory (section 4) (feat.data_model_change section 3.4; journey:feat.design NF-10).

`to_model` keeps its signature and its `(records, rejected)` return shape (feat.data_model_change section 4). `ingest.py`, `dq.py` and `pipeline.py` need no change (journey:feat.design HC-3, B1.1).

4. Derived mapping: Invoice records → Delay history

Source (Invoice)	Target	Rule	Source
`paid`, `paid_date`	Filter	Only records where `paid` is truthy and `paid_date` is not `None`	feat.data_model_change (section 3.4); journey:feat.design (B1.2 `delays.py`)
`vendor`	Key of `Dict[str, List[int]]`	Vendors in first-seen order; a vendor with no qualifying record has no key	feat.data_model_change (section 3.4); journey:feat.design (B1.5)
`paid_date`, `due_date`	One `delay_days` value	`(paid_date - due_date).days`, calendar days; negative values (early payment) kept; values in feed order	feat.data_model_change (section 3.4); record:memory (D-2026-031)
—	Persistence	None: in memory only; the input records are not modified	feat.data_model_change (sections 1, 3.4); journey:feat.design (B1.2)
—	Consumer	ap_forecast `vendor_risk(history)`, one vendor's list at a time (DC-2)	feat.data_model_change (section 3.4); journey:feat.design (DC-2)

ap_aging (DC-1 consumer) reads `id`, `vendor`, `due_date`, `amount` and `paid` and ignores the extra `paid_date` key, so the new mapping does not affect it (feat.data_model_change section 4; journey:feat.design DC-1).

5. Schedule mapping

Attribute	Customer contract	`schedule.json` today	Change	Source
Frequency	daily	`"frequency": "daily"`	None	org.customer_mappings; tool:repo (`schedule.json`)
Time	02:00	`"time": "02:00"`	None	org.customer_mappings; tool:repo (`schedule.json`)
Timezone	Asia/Kolkata	`"timezone": "Asia/Kolkata"`	None	org.customer_mappings; tool:repo (`schedule.json`)

This step changes no schedule. It records that the customer frequency and the pipeline config agree (org.customer_mappings; tool:repo `schedule.json`). DE-13 Scheduling the pipelines reads org.customer_mappings to set the schedule (file:hobasa-data-engineering-agent/.claude/skills/data-engineering-steps/SKILL.md).

6. Mapping validation cases (for coding and unit tests)

DE-08 Coding reads feat.mappings, and DE-09 Unit Test Cases writes the unit tests for the transformation logic (file:hobasa-data-engineering-agent/.claude/skills/data-engineering-steps/SKILL.md). Test rows use synthetic data, as the design requires for tests and examples (journey:feat.design SE-4).

Case	Expected	Source
v2 header (no `paid_date` column), valid row	Loaded; `paid_date is None`	feat.data_model_change (section 4); journey:feat.design (B1.5 `test_feed_v2_without_paid_date_column`)
v3 row, `paid_date` empty (unpaid, as the contract sends it)	Loaded; `paid_date is None`	org.customer_mappings; journey:feat.design (B1.5 `test_paid_date_empty_is_none`)
v3 paid row with a valid ISO `paid_date`	Loaded; `paid_date` is a `date`	org.customer_mappings; journey:feat.design (B1.5 `test_paid_date_parsed_for_paid_row`)
v3 row with non-blank invalid `paid_date` (`"2026-13-01"`, `"not-a-date"`)	Rejected, reason `'paid_date'`; `reject_reasons` lists it	journey:feat.design (B1.4, B1.5 `test_invalid_paid_date_rejected`)
Bad `due_date` and bad `paid_date` on one row	Rejected, reason `ValueError`	journey:feat.design (B1.5 `test_existing_reason_wins`, DD-6)
Unpaid row with a valid `paid_date`	Accepted; date kept	journey:feat.design (B1.5 `test_unpaid_row_keeps_paid_date`, DD-7)
Every accepted record	Keys `id`, `vendor`, `due_date`, `amount`, `paid`, `paid_date`; `amount` is `Decimal`	feat.data_model_change (section 3.2); record:memory (D-2026-044)
Existing tests	`test_run_maps_and_rejects`, `test_paid_flag`, `test_missing_column_rejected` pass unchanged	tool:repo (`tests/test_pipeline.py`); journey:feat.design (B1.5)

The repository's sample feed has the v2 header `invoice_id,vendor,due_date,amount,paid`, so it covers the v2 path only; v3 cases need synthetic rows in the tests (file:ap-pipeline-sample/data/sample_feed.csv).

7. Code in this step

This step updates the mapping document only and changes no code. The tool view shows `transform.py` without `paid_date` and no `delays.py` module (tool:repo). TSK-01 (`to_model` with `_parse_paid_date` and `PAID_DATE_REASON`) and TSK-02 (`delays.delays_by_vendor`) implement this mapping in DE-08. At that point the `transform.py` module docstring, which lists the model as "(id, vendor, due_date, amount, paid)", should also name `paid_date` (tool:repo `transform.py`; feat.data_model_change section 5).

8. Open questions (non-blocking)

Proposed owners are this step's proposal, except where the cited item names the owner.

ID	Question	Interim position	Owner (proposed)	Source
MQ-1	CQ-DE4-4 (carried) said no feed v3 specification was in the inputs. The customer contract now lists the v3 columns and names `paid_date`. Can CQ-DE4-4 be closed?	Yes; this mapping uses `paid_date`	Data modeler	feat.data_model_change (section 7, CQ-DE4-4); org.customer_mappings

ID	Question	Interim position	Owner (proposed)	Source
MQ-2	The contract says `paid_date` is "empty when unpaid", but the model accepts an unpaid row with a valid `paid_date` and keeps the date (DD-7, OQ-B6). Should such a row instead be rejected, or its date cleared?	Keep it, as the model says; the Delay history leaves it out because it filters on `paid`	Product manager (OQ-B6); feed owner	feat.data_model_change (sections 3.2, 3.4, 7 OQ-B6); journey:feat.design (DD-7); org.customer_mappings
MQ-3	The contract names feed v3 but gives no effective date, and CQ-DE4-1 (when v3 reaches production) is still open. Must v2 and v3 feeds load side by side?	The mapping accepts both, because a missing column gives `None`	Product manager / Data engineering lead	feat.data_model_change (sections 4, 7 CQ-DE4-1); org.customer_mappings
MQ-4	Is a `paid_date` cell that holds only whitespace "blank" (→ `None`) or invalid (→ reject with `'paid_date'`)? The inputs say "blank" → `None` and "parsed with `date.fromisoformat` after stripping whitespace", but they do not name this case or the order of the two checks	Unconfirmed; DE-08 must not pick a behaviour silently and should cover the chosen behaviour with a unit test once the data modeler answers	Data modeler	feat.data_model_change (section 3.2); journey:feat.design (B1.2)
DMQ-1, DMQ-2, DMQ-3 (carried)	`vendor` optional or required; who updates o `rg/org.data_model.md`; `amount` 2 dp not enforced	Unchanged; this mapping does not change `vendor` or `amount`	Data modeler; Data engineering lead	feat.data_model_change (section 7)

No blocking questions.

9. Validation check

Check	Result	Source
Mapping updated from the data model	Pass: every Invoice attribute in feat.data_model_change section 3.1 has one source column (section 2); the Delay history is mapped (section 4)	feat.data_model_change
Latest customer mapping applied	Pass: v3 columns and `paid_date -> Invoice.paid_date` (sections 1, 2); schedule confirmed (section 5)	org.customer_mappings
Differences between the Data Model change, the design and the customer contract	One wording difference, raised as MQ-2; no other difference	org.customer_mappings; feat.data_model_change; journey:feat.design
Decisions and lessons	Decimal amounts (D-2026-044, L-011); calendar-day delay (D-2026-031)	record:memory
Test data	Validation cases use synthetic rows (section 6)	journey:feat.design (SE-4)